Holodeck: A VR-First System for Prompt-Driven Walkable World Generation

Leslie Osei-Anane

April 9, 2026

1 Introduction

Science fiction has always been reliable at predicting what we want before we know how to build it. The communicator from Star Trek became the first commercial flip phone. The tablet computer from 2001: A Space Odyssey became the iPad. These weren't coincidences. They were the same human desire, expressed first in story, then in silicon.

Of all the technologies science fiction imagined, the Holodeck stuck around the longest without a real answer. A user walked in, described a world, and the system built one around them. No menus, no asset browsers, no friction between imagination and experience. It felt further away than flying cars because the problem it was solving was harder to name. The hardware was the easier problem to solve. Understanding what a person meant, building something coherent from it, and delivering it before the moment of anticipation turned into waiting was the actual problem. That sequence of steps is deceptively demanding. A user speaks a sentence. The system interprets intent, selects a layout, furnishes it with appropriate objects, themes it correctly, and verifies that every surface is safe to walk on. In VR specifically, all of that has to work before the user takes a single step. The system earns the experience before it offers it.

Recent work has made real progress on pieces of this. ProcTHOR demonstrates that large-scale valid indoor environments can be generated procedurally with measurable generalization benefits for embodied AI [2]. Text2Room shows that a text prompt can seed a spatially consistent 3D mesh by lifting 2D generative outputs into a 3D representation [5]. Work directly named Holodeck has explored language-guided 3D environment generation by selecting and placing assets to satisfy language-specified constraints [7]. Each of these advances a different part of the problem. None of them put a user inside a headset and asked the system to perform in real time.

This paper describes a VR-first system in which a user enters a stable origin room, submits a natural language prompt by voice or text, and steps through a portal into a compiled, walkable destination world. The client application is implemented in Unity using C# and targets the Meta Quest platform via the OpenXR runtime. Prompts are converted into a validated intermediate representation called WorldSpec, which a deterministic compiler transforms into a runtime-executable world bundle. To reduce perceived latency and prevent disorienting world-formation effects, the destination world appears first as a walkable greybox

and upgrades visually in streaming phases as assets load. The system is evaluated on time to first walkable world, compilation success rate, constraint satisfaction, and runtime performance under Quest-native conditions.

2 Background

2.1 Large Language Models and Structured Output

Large language models (LLMs) are neural networks trained on large corpora of text to model and generate natural language. When prompted with appropriate constraints, these models produce outputs that conform to structured formats, including JSON schemas, enabling their integration as planning components in software pipelines. Prior work has demonstrated this planning pattern by mapping natural language into structured layout representations suitable for downstream scene construction [3]. Schema-locking, which constrains model output to a predefined JSON schema, reduces the likelihood of hallucinated or invalid field values and allows downstream components to parse and validate model responses deterministically. This property is central to the system described in this paper, which uses an LLM to convert natural language prompts into a validated WorldSpec representation rather than generating 3D content directly. A deterministic fallback planner handles cases where the LLM produces invalid or out-of-registry outputs, ensuring the pipeline succeeds even under model failure.

2.2 Virtual Reality and OpenXR

Virtual reality systems present stereoscopic rendered environments through a head-mounted display, tracking head and controller motion to update the view in real time. The OpenXR standard defines a cross-platform API for extended reality applications, allowing a single codebase to target multiple headset platforms [6]. The Meta Quest line of headsets runs OpenXR natively and imposes strict performance constraints: frame time budgets, draw call limits, and texture memory ceilings must be respected to maintain visual stability and user comfort. Teleport locomotion, a standard interaction pattern in VR, allows a user to select a destination on a valid floor surface and reposition instantly, sidestepping continuous motion that can cause discomfort. These constraints shaped the design of this system at every level, from asset budget enforcement to the decision to prioritize greybox compilation before visual quality.

2.3 Asset-Based Scene Composition

Asset-based scene composition constructs environments by selecting and placing pre-authored 3D objects from a controlled library, rather than generating geometry procedurally from noise functions or lifting it from 2D outputs. This approach trades generative range for reliability: the resulting environments contain assets with known geometry, collision data, and performance characteristics that a runtime can depend on. Large-scale procedural environment generation systems such as ProcTHOR illustrate this trade-off by composing valid scenes from constrained asset sets and layout rules [2]. Procedural placement systems

arrange these assets according to spatial constraints and layout rules while preserving physical validity. This system adopts asset-based composition as its primary generation strategy, supplemented by procedurally generated greybox geometry for structural layout. Assets enter the planner-safe inventory only after passing a manual review pipeline that validates semantic quality and assigns exclusion flags where appropriate.

2.4 Compiler Pipelines

Compiler pipelines transform high-level input into validated intermediate representations before producing executable outputs. Classic compiler design formalizes this staged structure as lexical and syntactic analysis, semantic checks, intermediate representation generation, optimization, and target code emission [1]. The same architectural pattern is useful for prompt-to-world systems: natural language is parsed into a constrained WorldSpec, validated for schema and safety constraints, and then compiled into runtime-ready artifacts. Treating world generation as a compiler pipeline makes failures diagnosable at stage boundaries, supports deterministic fallback behavior, and reduces runtime surprises in VR.

3 Related Work

Research in prompt-driven 3D environment generation has split along one foundational question: should a system generate novel geometry from a prompt, or assemble environments from a curated library of pre-authored assets? That choice has consequences for reliability, performance, and what guarantees a downstream runtime can actually make. Text2Room takes the generative geometry approach. Hollein et al. showed that a text prompt can produce a spatially consistent, textured 3D mesh by lifting outputs from 2D image generation models into a 3D representation [5]. The strength of this approach is coverage. Because geometry is synthesized rather than retrieved, the system can respond to arbitrary prompts without being limited by an asset catalog. In a live VR context, however, synthesized geometry requires post-generation analysis to determine walkability, draw call cost, and memory footprint before a runtime can safely load it. That additional step adds latency and introduces failure modes at the validation stage that are difficult to recover from within an interactive session. 3D-SceneDreamer addresses one dimension of this problem by representing global scene structure in a stable tri-plane NeRF and refining content in stages [9]. The staged approach improves cross-view consistency and reduces cumulative geometric error, and that architectural pattern, stabilizing global structure first then refining locally, maps onto the greybox-first compilation strategy this system uses. The difference is that geometry synthesis still produces outputs without known collision or performance properties, whereas asset-based compilation carries those properties from the start.

The asset-based approaches accept a narrower generative range in exchange for stronger guarantees. Deitke et al. and Yang et al. both work within pre-authored asset libraries, using structured representations and constraint satisfaction to produce environments whose physical properties are known before the world loads [2, 7]. Deitke et al. focus on procedural layout generation at scale, producing large collections of varied indoor environments for embodied AI training [2]. Yang et al. connect natural language to asset selection and

spatial placement directly, using a language model to convert prompts into structured scene descriptions that get compiled into interactive environments [7]. Both systems treat generation as a pipeline with explicit validation stages, and both produce environments reliable enough to train agents in. Both systems assume the asset library contains what the prompt requests. Asset coverage is an implicit prerequisite, not a problem the pipeline resolves. Feng et al. extend this direction by showing that large language models can serve as visual planners, converting natural language prompts into stylesheet-like structured layouts through in-context demonstrations [3]. Their results support a clear separation of concerns: planning in structured symbolic space first, then compilation afterward. That separation directly motivates the WorldSpec stage in this system, where the language model produces a schema-locked intermediate representation rather than geometry, and a deterministic compiler handles the rest.

A related body of work approaches structured scene representation through scene graphs, where objects are nodes and spatial relationships are edges. GraphDreamer uses scene graph conditioning to generate compositional 3D scenes, introducing object-wise disentanglement and relationship-aware constraints to reduce interpenetration and improve spatial coherence [4]. CommonScenes takes a similar position, combining a layout branch and a shape generation branch conditioned on scene-object and object-object relationships, and reports improved semantic consistency compared with retrieval-heavy alternatives [8]. Both works provide evidence that explicit relational structure, represented before rendering rather than inferred from it, produces more controllable and reproducible outputs. For the purposes of this system, the scene graph functions as an analogue to WorldSpec: a machine-checkable intermediate representation that the compiler can validate against spatial and budget constraints before any geometry is loaded.

All three systems treat generation as an offline or training-time problem, where compilation time and frame stability fall outside the design requirements. A user standing in a headset waiting for a portal to open operates under a different set of conditions. The system described in this paper sits within the asset-based tradition of Deitke et al. and Yang et al., adopting structured intermediate representations and constraint-driven compilation for the same reliability reasons, but applies them to a live interactive setting where those guarantees have to hold in real time [2, 7]. A deterministic fallback planner handles cases where the language model produces invalid output. A greybox-first compilation phase ensures a walkable world exists before visual assets begin streaming. A substitution system replaces missing assets with the closest tagged alternatives, keeping the pipeline moving regardless of asset coverage gaps. The contribution is a composition of these properties into a pipeline that maintains reliability guarantees under the constraints of a standalone VR headset.

4 Methodology

4.1 Client (VR application)

The client consists of the Unity-based VR runtime, the Creation Room, the in-world assistant panel, the portal transition system, and the destination-world executor. Together, these components handle prompt entry, user feedback, comfort-preserving movement, and the

final handoff into the compiled world. The client enforces the runtime interaction contract by reading backend-produced manifest and phase artifacts, preparing the portal only after a valid destination is available, and relocating the XR origin to a validated safe spawn point on entry. It also consumes backend scene context directly: room archetype, decor plan, exact asset realizations, and validated transforms are all passed in rather than improvised on the headset. At runtime, the client applies comfort and presentation logic such as teleport-surface enforcement, collision-aware destination execution, destination-world polish, and return-home transitions while keeping headset-side behavior predictable.

4.2 Backend (planning, compilation, and validation)

The backend consists of the planner, WorldSpec validator, pack and stylekit registries, substitution layer, safe-spawn search, decor planner, runtime realization registry, and the phase0 compiler that writes the runtime contract. Together, these components convert a prompt into a validated manifest and phase artifact that the Unity client can execute predictably.

4.2.1 Backend Pipeline

Planning occurs in two stages. First, the language model converts the prompt into a structured intent and selection proposal over a safe, indexed asset shortlist, producing a schema-constrained WorldSpec rather than geometry directly [3, 7]. That intent stage asks the model for a fuller scene program: room archetype, required and optional roles, role counts, a primary anchor object, supporting objects, a relation graph, density target, symmetry preference, walkway intent, and style cues. If that semantic step fails or returns invalid identifiers, the request fails explicitly.

Compilation turns that validated WorldSpec into something Unity can actually use. The compiler resolves missing assets through constrained substitution, builds the destination shell, accepts model-proposed object placements, computes a safe spawn, and writes the manifest and phase0 artifact that the client reads at runtime. The runtime contract carries more than raw transforms. It also includes explicit roles, scene context, and a decor plan so the room can be realized as a coherent space rather than as a loose set of objects. Validation runs through the whole process: schema checks catch malformed WorldSpecs, semantic checks catch missing required roles, post-placement checks catch overlap and entry problems, and budget checks help preserve Quest-compatible performance before the portal is allowed to open.

This split matters because a single "generate everything" step makes it hard to tell what actually went wrong. A bad result can come from prompt interpretation, asset selection, placement, or runtime realization. Breaking the work into stages makes those failures visible and makes debugging more concrete.

4.2.2 Semantic Planning

The semantic planning stage is built around a structured scene program rather than a free-form description. In practice, the planner asks the language model for a room archetype,

required roles, optional roles, role counts, a primary anchor object, secondary support objects, a relation graph, density, symmetry, walkway intent, scene features, and broad style cues [3, 4, 8]. That scene program functions as the semantic contract for the rest of the backend.

This stage became necessary because prompt text by itself was too ambiguous to carry safely through the compiler. Earlier attempts leaned too heavily on prompt parsing and small heuristics, which made the planner brittle and encouraged prompt-specific fixes inside later stages. A prompt such as "a cozy reading room with rain outside the windows" mixes mood, objects, and scene cues together. If those signals stay as raw text, later parts of the pipeline have to guess what matters.

The scene-program representation fixes that by normalizing the model output before it is accepted. Unsupported roles are removed, malformed relations are rejected, and valid archetype labels are trusted only when they match the project's fixed ontology. This follows the same general lesson seen in LayoutGPT and Holodeck: the model should describe what the room should contain and how the parts relate, while later layers decide whether the room can actually be built [3, 7]. In practice, this let the backend drop several earlier heuristics, including hand-written role injection for otherwise valid model outputs.

4.2.3 Approved Asset Pipeline

The approved asset pipeline is the reviewed inventory that the planner and runtime are allowed to use. Prefabs are scanned, enriched with Unity-side metrics, and then placed in review queues where assets can be approved, excluded, or skipped. Semantic overrides are stored separately so that an approved asset is not automatically trusted as the right answer for every role. After review, the backend builds a planner-safe pool and a runtime realization registry from the approved set only, following the same general asset-constrained direction seen in ProcTHOR and Holodeck [2, 7].

This pipeline became necessary because the raw Unity project was too noisy to use directly. Asset names often contain misleading tokens, and some prefabs that look semantically useful in text are poor matches in practice. A prefab called "DeskPlant" may not be an appropriate desk, and a "TableNumber" prop should not satisfy a required table role just because the word "table" appears in its name. In a Quest-focused VR project, mistakes of that kind become visible quickly because scale, collision, and material problems all reach the user directly.

The approved asset pipeline addresses this by separating two different questions: whether an asset is allowed in the system, and what role that asset is allowed to satisfy. The planner therefore searches a bounded inventory whose assets have already passed at least a minimal semantic and performance check, rather than searching every prefab in the repository. That narrower operating space strongly improved selection quality and reduced cases where the room was technically compilable but obviously wrong to a human observer.

4.3 Implementation

The most important design choices are to use an LLM for semantic planning and direct placement, while keeping validation, runtime safety, and asset eligibility explicit in the

compiler. A validated room-shell compile stage, approved-only asset selection, explicit safe-spawn checks, and a backend-generated runtime realization registry were chosen to preserve reliability and comfort on standalone VR hardware. In practice this means the language model proposes both what the room should be and where the chosen objects should go, while the backend decides whether that proposal is acceptable for VR use.

Early tests made it clear why spatial awareness mattered. Even when the selected assets were broadly correct, the room could still look wrong if objects were dropped into open space without much regard for one another. Desks, chairs, and other props read less like a furnished room and more like a pile of disconnected objects. That problem pushed the project toward relation-aware placement, stronger scene context, and stricter compile-time checks.

Figure 1: Room output before spatial placement was treated seriously. The selected assets are present, but the layout reads more like objects dumped into a shell than a coherent room.

The implementation work therefore moved in two directions at once. The planner became more structured and less ad hoc, while the compiler became stricter and more explicit about what counted as a valid room. Those changes needed each other. A stronger planner without a careful compiler would still allow unsafe or awkward layouts. A strict compiler without a meaningful planner would reject too many interesting prompts. The system only became usable once both sides improved together.

4.3.1 How Spatial Placement Was Implemented

Spatial placement is handled as direct coordinate prediction from the language model after semantic planning and asset selection are complete. The backend first builds a room shell and converts each selected object into a simplified geometry profile. The language model then places those approved objects numerically inside the room, using the scene program as context rather than working from the raw prompt alone. This still follows the broad pattern emphasized by LayoutGPT and Holodeck, where semantic structure is made explicit before

final world construction [3, 7]. There is no secondary placement solver that rearranges the room afterward. The model decides the arrangement, and the compiler either accepts that proposal or rejects it.

The richer scene program gives that placement stage more direction than a flat role list did. It specifies an anchor object, support hierarchy, relation graph, density target, and walkway-preservation hints, so the placement request is much closer to room composition than to a simple list of nouns. GraphDreamer and CommonScenes both argue that explicit relational structure improves scene coherence [4, 8], and that same idea is used here to give the model more meaningful spatial work. The compiler still checks the result afterward for safety and runtime suitability, but the arrangement itself comes from the model.

4.3.2 Safe Spawn and Portal Readiness

One of the simplest but most important implementation details is the safe-spawn search. After the room shell and placements are compiled, the backend searches for a valid player spawn by testing candidate points on the walkable floor while avoiding occupied space. The portal is not allowed to open unless this step succeeds. That means a room can be semantically valid and still be rejected if it does not contain a safe place to enter.

This gate is part of what makes the architecture feel more like a compiler than a demo script. The system does not assume that a plausible room is good enough. It proves that the player can enter it safely, that the floor is valid for teleport locomotion, and that the backend can produce a complete runtime contract before the Unity client commits to the transition.

This check matters for both comfort and credibility. From a comfort standpoint, the user should never emerge from the portal inside a chair, too close to a wall, or facing a cluttered corner with no space to move. From a systems standpoint, safe spawn is the final proof that the room is not only a semantic answer to the prompt but also a usable destination. In that sense, spawn selection functions as a practical end condition for the compiler: if a safe spawn cannot be found, the room is not ready.

4.4 Research Questions and Goals

My research question is whether a prompt-driven system can convert natural language into a walkable VR world quickly enough to remain interactive while still enforcing safety, validity, and runtime constraints. The main goals are to generate a validated WorldSpec from a prompt, compile it into a room the user can safely enter, and measure whether this pipeline remains reliable on standalone VR hardware.

4.5 Data, Tools, and Systems

The system uses manually reviewed asset packs, stylekit manifests, WorldSpec and manifest JSON contracts, and compiled phase0 artifacts as its primary data sources. The implementation uses Unity with C# and OpenXR on the client side, and Python modules for planning, validation, substitution, safe-spawn search, and compilation on the backend, with Gemini providing the semantic planning path.

4.6 Replication Requirements

To replicate the system, a developer would need the Unity project, the Python backend, the reviewed asset and stylekit manifests, a configured '.env' file for the LLM provider, and the dependencies listed in 'requirements.txt'. They would also need a Quest-compatible OpenXR setup for runtime testing, although the backend contract generation can be reproduced from the terminal without the headset.

5 Results and Findings

This section will show what the system actually produced during testing, including whether prompts turned into valid room contracts, whether compilation succeeded, and how long it took before a walkable room was ready. It will also look at whether the generated rooms matched the general intent of the prompt and whether the portal, safe spawn, and placement checks worked consistently.

6 Discussion

This section will talk about what worked, what still feels limited, and where the tradeoffs showed up most clearly in the project. In particular, it will reflect on the decision to let the LLM handle meaning while keeping validation, placement, and runtime safety deterministic, and it will discuss current limits such as asset coverage, provider reliability, and visual richness.

7 Conclusion

This system currently targets indoor room-scale environments compiled from a fixed, manually reviewed asset library. Several directions remain open for future work. Asset coverage is the most immediate constraint: expanding the indexed inventory through additional reviewed packs would allow the planner to satisfy a broader range of prompts without falling back to substitution. Supporting multi-room and outdoor environments would require extending the WorldSpec schema and the greybox compiler to handle more complex spatial graphs. Geometry synthesis, as demonstrated by approaches such as Text2Room [5], represents a longer-term direction for handling prompts that fall entirely outside the indexed inventory, provided that synthesized geometry can be validated against the same walkability and performance constraints the current compiler enforces. Finally, the progressive streaming architecture opens the door to interactive world editing, where a user could modify a StyleKit or adjust prop density on an already-compiled world without triggering a full recompilation.

References

[1] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*, 2 ed. Addison-Wesley, 2006.

[2] Deitke, M., VanderBilt, E., Herrasti, A., Weihs, L., Ehsani, K., Salvador, J., Han, W., Kolve, E., Kembhavi, A., and Mottaghi, R. ProcTHOR: Large-Scale Embodied AI Using Procedural Generation. In *Advances in Neural Information Processing Systems* (2022), vol. 35, Curran Associates, Inc.

[3] Feng, W., Zhu, W., Fu, T.-J., Jampani, V., Akula, A., He, X., Basu, S., Wang, X. E., and Wang, W. Y. LayoutGPT: Compositional visual planning and generation with large language models. In *Advances in Neural Information Processing Systems* (2023), vol. 36, Curran Associates, Inc.

[4] Gao, G., Liu, W., Chen, A., Geiger, A., and Schölkopf, B. GraphDreamer: Compositional 3D scene synthesis from scene graphs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 2024), IEEE/CVF, pp. 21295–21304.

[5] Höllein, L., Cao, A., Owens, A., Johnson, J., and Niessner, M. Text2Room: Extracting Textured 3D Meshes from 2D Text to Image Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (October 2023), IEEE/CVF, pp. 7909–7920.

[6] Khronos Group. OpenXR 1.1 specification. URL: https://registry.khronos.org/OpenXR/specs/1.1/html/xrspec.html, 2024. Accessed: March 5, 2026.

[7] Yang, Y., Sun, F.-Y., Weihs, L., VanderBilt, E., Herrasti, A., Han, W., Wu, J., Haber, N., Krishna, R., Liu, L., Callison-Burch, C., Yatskar, M., Kembhavi, A., and Clark, C. Holodeck: Language Guided Generation of 3D Embodied AI Environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 2024), IEEE/CVF, pp. 16227–16237.

[8] Zhai, G., Örnek, E. P., Wu, S.-C., Di, Y., Tombari, F., Navab, N., and Busam, B. CommonScenes: Generating commonsense 3D indoor scenes with scene graph diffusion. In *Advances in Neural Information Processing Systems* (2023), vol. 36, Curran Associates, Inc.

[9] Zhang, S., Zhang, Y., Zheng, Q., Ma, R., Hua, W., Bao, H., Xu, W., and Zou, C. 3D-SceneDreamer: Text-driven 3D-consistent scene generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 2024), IEEE/CVF, pp. 10170–10180.